

Smart sensing: The system always knows your current speaking level. When you succeed (3 “perfect” in a row), it auto-advances to the next level.

Indexed scaling: Thousands of prompts are instantly accessible by scenario/level—expand the RESPONSES array as needed.

Talking AI: Every prompt has a Speak button powered by TTS.

Strict correction feedback: After each response, the AI predicts and issues a correction according to your ordered priorities, only one at a time. Perfect attempts are recognized.

Progress tracking: Students can pause/resume; their place, level and progress are saved.

****Student can change scenario or reset at any time.**

Smart ai persona

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Smart AI Persona Roleplay Trainer</title>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <style>
    body { font-family: Arial, sans-serif; background:#f9fcfe; margin:2em auto; max-width:700px; }
    h1 { color: #23447a;}
    .card { background:#fff; border-radius:8px; padding:1.3em 1.2em; margin-bottom:2em;
box-shadow:0 1px 8px #0001;}
    .prompt-block { background:#f3f7fc; border-radius:8px; padding:1em; margin-bottom:1.1em;
display:flex; align-items:center; }
    .prompt-txt { flex:1; }
    .speak-btn { margin-left:1em; background:#23447a;color:#fff;padding:0.5em 1.1em;border:none;
border-radius:4px; cursor:pointer;}
    .speak-btn:hover {background:#172e4d;}
    .progress-bar {background:#e5ebf4; border-radius:6px; width:100%;margin:0.3em 0; height:14px;
position:relative;}
    .prog-fill {background:#2fa37b;height:100%; border-radius:6px;}
    .progress-label {font-size:0.93em;float:right;}
    #userAudio {margin:0.8em 0;}
    button, select {font-size:1em;}
    .complete { color:#179b41;font-weight:bold; }
    .ai-correction { background:#faf7e7; border-radius:6px; padding:0.6em 1em; margin-top:0.7em; }
  </style>
</head>
<body>
<h1>Smart AI Persona – Speaking Roleplay</h1>
<div class="card" id="userInit"></div>
<div class="card" id="mainApp" style="display:none;">
```

[illegible]

```

    { issue: "Breath control", cmd: "Control your breath.", phrase: "mmm, zzz" }
];

// === Indexed structure for O(1) lookup ===
const index = {};
RESPONSES.forEach(r=>{
  let k = `${r.scenario}_${r.level}_${r.role}`;
  if(!index[k]) index[k]=[];
  index[k].push(r);
});

// === Local storage state ===
function saveState(obj){ localStorage.setItem("smartai_state",JSON.stringify(obj)); }
function loadState(){ return JSON.parse(localStorage.getItem("smartai_state"))||"null"; }

// === User boot/init ===
const levels = ["A1","A2","B1","B2","C1","C2"];
const scenarios = Array.from(new Set(RESPONSES.map(r=>r.scenario)));

function setupUserInit() {
  let st = loadState()||{};
  let html = `
    <b>Choose your starting speaking level:</b>
    <select id="iniLevel">${levels.map(l=><option>${l}</option>`)}</select>
    <b>Choose scenario:</b>
    <select id="iniScenario">${scenarios.map(s=><option>${s}</option>`)}</select>
    <button onclick="startApp()">Start</button>
    ${st.username ? `<hr><div><b>Welcome back,</b> Level: ${st.level}, Scenario: ${st.scenario}</div>` : ''}
    <button onclick="resumeApp()">Resume</button>
    <button onclick="localStorage.clear();location.reload();" style="float:right;">Reset All</button>`.trim()
  `;
  document.getElementById("userInit").innerHTML=html;
}
setupUserInit();

function startApp() {
  let level = document.getElementById("iniLevel").value;
  let scenario = document.getElementById("iniScenario").value;
  let state = {level, scenario, curPrompt:0, progress: [], correctStreak:0};
  saveState(state);
  showMainApp();
}
function resumeApp(){
  showMainApp();
}

```

```
}
```

```
// === AI Talking ===
```

```
function speak(text){  
  if(!window.speechSynthesis) return;  
  let u = new SpeechSynthesisUtterance(text);  
  u.lang = 'en-US'; u.rate=1.03; u.pitch=1.03;  
  // Optional: set preferred voice  
  window.speechSynthesis.speak(u);  
}
```

```
// === Main App Loop ===
```

```
function showMainApp(){  
  document.getElementById("userInit").style.display = "none";  
  document.getElementById("mainApp").style.display = "";  
  renderMain();  
}  
function renderMain() {  
  let st = loadState();  
  document.getElementById("curLevel").textContent = st.level;  
  document.getElementById("curScenario").textContent = st.scenario;
```

```
  // Get all prompts for this scenario/level/role=system  
  let key = `${st.scenario}_${st.level}_system`;  
  let prompts = index[key]||[];  
  let progress = st.progress||[]; // array of completed prompt indices
```

```
  // Progress visuals  
  let pct = prompts.length ? Math.round(progress.length*100/prompts.length): 0;  
  document.getElementById("progFill").style.width = pct+"%";  
  document.getElementById("curProg").textContent = `${progress.length}/${prompts.length}  
  complete`;
```

```
  // What is next prompt? (skip completed)  
  let nextIdx = progress.length;  
  let done = nextIdx>=prompts.length;  
  let html = "";
```

```
  // If all prompts for this level done, offer move up!  
  if(done){  
    let curLvIdx = levels.indexOf(st.level);  
    let canUpgrade = curLvIdx+1<levels.length &&  
    index[`${st.scenario}_${levels[curLvIdx+1]}_system`];  
    html = `<div class="prompt-block complete">
```

```

    <b>Level Complete!</b><br>
    ${canUpgrade?'<button onclick="upgradeLevel()">Go to next level
(+levels[curLvIdx+1]+'>')</button>:'}
    <button onclick="switchScenario()">Switch Scenario</button>
</div>`;
document.getElementById("nextPromptBox").innerHTML = html;
document.getElementById("aiCorrectionBlock").innerHTML = "";
return;
}

let pltem = prompts[nextIdx];
html = `<div class="prompt-block">
    <div class="prompt-txt"><b>AI: </b>${pltem.prompt}</div>
    <button class="speak-btn" onclick="speak(\`${pltem.prompt.replace(/`/g,`\`)}\`)"><img alt="Speaker icon" data-bbox="760 328 783 345"/>
Speak</button>
</div>
<div>
    <label>Your response (recording simulated; enter text or notes):</label>
    <textarea id="studReply"></textarea>
    <button onclick="onSubmitAttempt()">Submit Attempt</button>
</div>
`;
document.getElementById("nextPromptBox").innerHTML = html;
document.getElementById("aiCorrectionBlock").innerHTML = "";
}
function onSubmitAttempt(){
    // Simulate "AI: listen" + correction logic
    let reply = document.getElementById("studReply").value.trim();
    if(!reply){ alert("Try to respond!"); return;}
    let st = loadState(), key = `${st.scenario}_${st.level}_system`, prompts = index[key]||[];
    // AI senses: choose main correction
    let correction = predictCorrection(reply);
    // Save result & increment progress
    st.progress = st.progress||[];
    st.progress.push(st.progress.length); // mark this index as done
    // Progress towards level up: if 3 in a row with few errors, suggest "move up"
    if(correction.perfect) st.correctStreak = (st.correctStreak||0)+1;
    else st.correctStreak = 0;
    saveState(st);
    document.getElementById("aiCorrectionBlock").innerHTML = `
    <div class="ai-correction">
    <b>AI Correction:</b><br>
    <span class="feedback-issue">Issue:</span> ${correction.issue}<br>
    <span class="feedback-cmd">Correction:</span> ${correction.cmd}<br>

```

```

    <span class="feedback-repeat">Repeat:</span> "${correction.phrase}" – ${correction.times}x
    ${correction.perfect?' <span class="complete">Perfect!</span>':''}
  </div>
  <button onclick="renderMain()">Next</button>
`;
    document.getElementById("nextPromptBox").innerHTML = "";
// Auto-level up after 3 perfect
    if(st.correctStreak>=3) setTimeout(=>autoLevelUp(),1200);
  }
function predictCorrection(reply){
  // Example: check for some common error patterns; can expand with real NLP/AI
  reply = reply.toLowerCase();
  if(/t$|k$d$p$|g$/.test(reply)) return {...CORRECT_PRIORITIES[0], times:3, perfect:true}; // has
  ending sound
  if(/quiet|soft|volume/.test(reply)) return {...CORRECT_PRIORITIES[1], times:3, perfect:false};
  if(/monotone|flat/.test(reply)) return {...CORRECT_PRIORITIES[2], times:3, perfect:false};
  if(/breath|breathe/.test(reply)) return {...CORRECT_PRIORITIES[3], times:3, perfect:false};
  // Else pick next unpracticed issue
  let rand = Math.floor(Math.random()*CORRECT_PRIORITIES.length);
  return {...CORRECT_PRIORITIES[rand], times:3, perfect:false};
}
function upgradeLevel(){
  let st = loadState();
  let idx = levels.indexOf(st.level);
  if(idx+1>=levels.length) return alert("Already at max level!");
  st.level=levels[idx+1]; st.progress=[]; st.correctStreak=0;
  saveState(st); renderMain();
}

```